

EVA calcule les valeurs *possibles* des variables d'un programme. Il est **prudent** : une alarme veut dire « *je n'arrive pas à prouver que c'est sûr* », et **pas** « *c'est faux* ». Ce TP est en **difficulté croissante** : vous avez déjà manipulé `-eva-slevel`, le déroulage et le widening au TP1 ; ici on rencontre des limites de plus en plus profondes d'EVA (on testera à chaque fois si ces réglages aident), et on finit par *documenter* une limite avec un contrat.

Méthode pour chaque exercice :

- (1) **Devine** le résultat attendu (sans lancer EVA).
- (2) **Lance** `frama-c -eva fichier.c` et **observe**.
- (3) **Compare** avec ta prédiction et **explique** l'écart.

Les fichiers source sont fournis. Une à deux phrases par question suffisent.

Exercice 1 — La garde qui ne protège pas [difficulté 1/5]

On veut éviter une division par zéro avec un test `if (d != 0)`.

```
int d = Frama_C_interval(-5, 5);    /* d peut valoir 0 */
int z = 0;
if (d != 0) {
    Frama_C_show_each_d_dans_le_if(d); /* que croit EVA pour d ICI ? */
    z = 100 / d;
}
```

1. Avec `if (d != 0)` juste avant `100/d`, EVA devrait être rassuré, non ?
2. Lance EVA. Regarde `Frama_C_show_each_d_dans_le_if` : quel intervalle EVA donne-t-il à `d` à l'intérieur du `if` ? Le 0 en est-il exclu ? L'alarme disparaît-elle ?
3. Essaie `-eva-slevel 20` (`slevel`, vu au TP1). Aide-t-il cette fois ?
4. Pourquoi EVA n'arrive-t-il pas à retirer *juste* le 0 ? *Indice* : un intervalle est « d'un seul tenant » — il ne peut pas avoir un trou au milieu.
5. Ouvre `ex1_garde_fix.c` : le test est réécrit en `if (d > 0) ... else if (d < 0) ...`. Lance EVA. Pourquoi ça marche ? La correction est-elle un *réglage* ou une *réécriture du code* ?

À retenir. Un intervalle ne peut pas exclure une valeur *au milieu* : la garde « différent de 0 » ne suffit pas, et `slevel` non plus. Il faut **réécrire** le test (cas positif / cas négatif) pour obtenir des intervalles propres.

Exercice 2 — Un pointeur qui vise deux variables [difficulté 2/5]

`p` pointe vers `x` ou `y` (au hasard) ; on met cette variable à 0 ; puis on divise par `x+y`.

```
int x = 1, y = 1;
int *p = Frama_C_interval(0,1) ? &x : &y;    /* p -> x OU y */
*p = 0;                                       /* une SEULE des deux passe
    a 0 */
int z = 100 / (x + y);
```

1. Après `*p = 0`, l'une vaut 0 et l'autre 1. Combien vaut `x+y` ? La division est-elle sûre ?

2. Lance EVA. Quelle alarme ? Qu'affiche-t-il pour `x` et `y` ?
3. EVA croit `x` et `y` peuvent valoir 0 *en même temps*. Pourquoi est-ce impossible en vrai ?
Indice : il suit x et y séparément et oublie qu'une seule a changé.
4. Relance avec `-eva-slevel 1`, puis 2. À partir de quelle valeur l'alarme disparaît-elle ? Pourquoi 2 précisément ?

À retenir. Pointeur imprécis → EVA met les deux cibles à « 0 ou 1 » et **perd la corrélation** « une seule a changé » → fausse alarme. Réparable par `slevel` (2 états suffisent).

Exercice 3 — Deux variables égales [difficulté 2/5]

On copie : `y = x`. Puis on teste `x == y`.

```
int x = Frama_C_interval(0,100);
int y = x;      int eq = (x == y);    /* vrai a tous les coups ? */
```

1. Puisque `y = x`, combien vaut `(x == y)` ?
2. Lance EVA *par défaut*. Que répond-il pour `eq` ? Est-ce exact ?
3. Relance avec `-eva-domains cvalue,octagon`. Réponse ?
4. Par défaut EVA suit `x` et `y` séparément (il « oublie » que `y = x`). En quoi le domaine des *octogones* (qui retient la relation `x - y`) change-t-il la donne ?

À retenir. Le domaine par défaut ne retient pas les *relations* entre variables : il échoue même sur `x == y` après `y = x`. Un domaine *relationnel* (octogones) garde la relation et répond juste.

Exercice 4 — Quand EVA calcule sur une adresse [difficulté 3/5]

On fabrique un indice à partir de l'adresse d'une variable (un « hash de pointeur »), puis on s'en sert pour écrire dans un tableau de 16 cases.

```
int *p = &x;
unsigned long a = (unsigned long)p;    /* l'adresse de x, vue comme un
entier */
unsigned long g = a % 16;               /* on la ramene entre 0 et 15 */
t[(int)g] = 1;                         /* ecriture sûre ? */
int b = Frama_C_interval(0,3000);     /* un VRAI nombre */
int c = b % 16;                       /* pour comparer */
```

1. `g = ... % 16` est forcément entre 0 et 15. `t[g]` est donc sûr, non ?
2. Lance EVA. Combien d'alarmes sur `t[(int)g]` ? Surpris ?
3. Que vaut `g` (le `% 16` d'une *adresse*) selon EVA ? Et `c` (le `% 16` d'un *nombre*) ?
4. Le `% 16` est le même. Pourquoi marche-t-il sur le nombre mais pas sur l'adresse ? *Indice : pour EVA, une adresse n'est pas un nombre mais un repère « (quel objet, quel décalage) ».*
5. Essaie `-eva-slevel 1000`. Le nombre d'alarmes change-t-il ?

À retenir. EVA ne sait pas calculer sur la *valeur numérique* d'une adresse : il produit une valeur floue (« garbled mix ») et lève de fausses alarmes. **Aucun réglage ne corrige** : limite *fondamentale*.

Exercice 5 — malloc dans une boucle [difficulté 4/5]

On construit une liste chaînée avec `malloc` dans une boucle, puis on additionne les valeurs des maillons.

```
int n = Frama_C_interval(1,1000); /* nombre de maillons, inconnu */
for (int i=0;i<n;i++){ node *m=malloc(sizeof(node)); if(!m) break;
    m->v=i; m->next=head; head=m; }
int s=0; for(node *c=head;c;c=c->next) s+=c->v; /* la somme */
```

1. La somme vaut $0 + 1 + \dots + (n - 1)$. Le programme a-t-il un bug?
2. Lance EVA sur `ex5_malloc.c`. Que vaut `somme`? Puis regarde la ligne `__malloc_w_main_17` : que vaut son champ `.v`? et `.next` (sur quoi pointe-t-il)? Combien d'objets EVA garde-t-il pour tous les maillons?
3. Change une seule chose à la fois :
 - `ex5_malloc_seul.c` (un seul `malloc`) : valeur précise? un `__malloc_w`?
 - `ex5_tableau.c` (un tableau, sans `malloc`) : un `__malloc_w`?
4. Le coupable est-il la boucle, le tableau, ou le `malloc` dans la boucle?
5. Essaie de réparer chacun. Sur `ex5_tableau.c` : `-eva-auto-loop-unroll 20`. Sur `ex5_malloc.c`, essaie les deux : `-eva-auto-loop-unroll 20`, puis `-eva-slevel 2000`. Lequel se répare? Lequel résiste à tout?

À retenir. `malloc` en boucle : EVA fond tous les maillons en un seul objet (« base faible »), `.next` pointant sur lui-même. Imprécision **fondamentale** (irréparable). Le tableau, lui, n'a qu'un *widening*, réparable par déroulage.

Exercice 6 — Documenter une précondition avec `requires` [difficulté synthèse]

La fonction `diviser(a,b)` n'est sûre que si $b > 0$. Plutôt que de tester partout, on l'annonce par un contrat.

```
/*@ requires b > 0; */
int diviser(int a, int b){ return a / b; }

int main(void){
    int ok      = diviser(100, 5); /* 5 > 0 : respecte la
        precondition */
    int d       = Frama_C_interval(-3, 3); /* d peut valoir 0 ou négatif */
    int risque  = diviser(100, d); /* precondition garantie ici ? */
}
```

1. Sans contrat, EVA peut-il prouver que `a / b` est toujours sûr?
2. Lance `frama-c -eva ex6_requires.c -main diviser` (la fonction seule, avec le `requires`). Reste-t-il une alarme de division?
3. Lance `frama-c -eva ex6_requires.c` (avec les appels). Regarde le statut de la *précondition*. Les appels `diviser(100,5)` et `diviser(100,d)` sont-ils logés à la même enseigne?
4. Le `requires` a-t-il supprimé le danger, ou l'a-t-il déplacé? Vers qui?

À retenir. Un `requires` ne fait pas disparaître le problème : il le documente et transfère l'obligation à l'appelant. C'est l'esprit des contrats ACSL.

Travail à rendre

Un compte-rendu PDF + les fichiers source modifiés. Pour chaque exercice, indique : (a) **faux positif ou vrai bug**, et pourquoi ; (b) si la limite est **réparable** (par quel moyen, à quel coût) ou **fondamentale**. Reste bref.